



MoveLab

KneeQuest — Gamified Knee-OA Rehabilitation System

A webcam-based, markerless motion-sensing home rehabilitation platform.
Practical roadmap to complete the remaining build: games, feedback,
database, dashboard, validation and deployment.

DOCUMENT	Remaining-Work Execution Plan
PREPARED FOR	Supervisor / Project Management Review
FLAGSHIP (TODAY)	Isometric Gamified Rehab Game — Wall-Sit Guardian
DATE	25 July 2026
STATUS	Sensing core complete · Game + data layers pending

1 Executive Summary

MoveLab is the software build for a Final-Year Project delivering a MediaPipe-guided, gamified home-rehabilitation system for knee osteoarthritis (KOA), to be evaluated in a randomised controlled trial against conventional physiotherapy. The core sensing technology — real-time, calibrated, clinically-meaningful knee-angle measurement from a standard webcam — is **built and exceeds the original design specification**. The remaining work converts that reliable angle stream into a playable, safe, data-logging rehabilitation product.

This plan breaks the outstanding work into small, executable tasks across eight tracks (Backend, Frontend, AI/Pose, Database, Dashboard, Testing, Documentation, Deployment), each with an objective, steps, dependencies, deliverable and effort estimate. The immediate priority (today) is the **Isometric Gamified Rehabilitation Game** — an isometric quadriceps hold game that reuses the existing pose pipeline and proves the end-to-end movement-to-game loop.

Target: a demonstrable MVP — sensing core + 1 isometric game + feedback + local database + basic dashboard — within ~4 working weeks, leaving the trial and thesis writing on schedule.

Indicator	Value	Note
Software MVP progress	~30–35 %	Sensing core done; game/data/feedback pending
Overall FYP progress	~15–20 %	Research/RCT track not yet started
Immediate deliverable	Isometric game	End-to-end movement-to-game loop (today)
MVP completion target	~4 weeks	Then validation → pilot → RCT

2 Current Project Status

2.1 Completed Modules

Module	Description	State
Pose loop	MediaPipe Pose (33 landmarks), web + desktop	Done
Knee-angle engine	View-adaptive 2-D sagittal / 3-D world angle	Done+
Signal conditioning	One-Euro filter, median outlier rejection, clamp	Done
Calibration	'Cal 0°' zeroing of extension offset	Done
Rep counter	Hysteresis state machine (straight↔bent)	Done
Live metrics	ROM min–max, per-knee confidence %	Done
Data export	CSV + JSON per-frame + summary	Done
App shell	PWA (manifest, installable), GitHub Pages deploy	Done

2.2 Remaining Work

Area	Outstanding item	Priority
Games	Isometric hold game (today) + flagship + 1–2 more	High
Coaching feedback	Corrective cues, tempo gate, trunk-lean detection	High
Database	SQLite schema: patients / sessions / scores / ROM / progress	High
Backend API	FastAPI service to persist sessions & serve dashboard	Med
Dashboard	Progress charts (ROM, reps, adherence) per week	Med
Progression logic	Auto difficulty rule (≥80% correct × 2 sessions)	Med
Safety layer	NPRS pain check, hold/depth caps, daily caps	High
Validation	Knee-angle vs goniometer (MAE, Bland–Altman)	Med
Documentation	README fix, user guide, methods chapter, API docs	Med
Deployment	PWA + backend packaging, release checklist	Med

2.3 Risks & Blockers

Risk / Blocker	Likelihood	Impact	Mitigation
Ethics approval delay	High	Critical path	Submit early; build software in parallel
Scope creep (10 games)	High	Med	Freeze MVP at 3 games; rest = future work
Two codebases (web + desktop)	Med	Med	Consolidate on web stack for MVP
Pose accuracy / lighting	Med	Med	Side-view guidance, confidence gating, validation
RCT recruitment	Med	High	Start screening during Month 4 pilot
Time for validation	Med	Low	Bundle goniometer study into testing week

3 Development Roadmap

Remaining work as small, executable tasks. IDs group by track. Effort in developer-days (d).

3.1 AI / Pose Estimation

AI-1 Isometric hold detection module

HIGH TODAY

Objective	Detect a stable knee-angle hold within a target band and accumulate hold time.
Steps	1. Add target band + tolerance config 2. Track time-in-band, reset on drift > tol 3. Emit hold-progress 0–1 + broken-hold event
Dependencies	Existing angle engine (index.html)
Deliverable	holdDetector.js with unit-testable pure function
Effort	0.5 d

AI-2 Coaching feedback engine

Objective	Convert angle/tempo/posture state into short corrective cues.
Steps	1. Define rule table (too shallow / slow / trunk lean) 2. Compute trunk angle from shoulder–hip line 3. Debounce cues to end-of-rep / end-of-hold
Dependencies	AI-1, trunk landmarks
Deliverable	feedback.js returning {cue, severity}
Effort	1 d

AI-3 Tempo gate

Objective	Reject reps/holds performed too fast to be controlled.
Steps	1. Time each phase 2. Flag < MIN_TIME as 'slow down' 3. Exclude flagged reps from scoring
Dependencies	AI-2
Deliverable	Tempo validation in scoring path
Effort	0.5 d

3.2 Frontend

FE-1 Game screen scaffold & router

HIGH TODAY

Objective	Add a screen system (menu → game → results) to the web app.
Steps	1. Add lightweight view router 2. Home / game / results containers 3. Wire existing tracker as a mode
Dependencies	None
Deliverable	Screen navigation in index.html
Effort	1 d

FE-2 Isometric game UI + canvas render

HIGH TODAY

Objective	Draw the game HUD, target band gauge and hold meter.
Steps	1. Hold gauge bound to AI-1 progress 2. Target band + live angle overlay 3. Timer, score, streak HUD
Dependencies	FE-1, AI-1
Deliverable	game_isometric.js render layer
Effort	1 d

FE-3 Results screen

Objective	Show post-session summary and export.
Steps	1. Hold time, best hold, stars, correct % 2. Reuse chart + CSV/JSON export 3. Save-to-history hook
Dependencies	FE-2, DB-1
Deliverable	Results view + export
Effort	0.5 d

3.3 Backend

BE-1 Local session API (FastAPI)

Objective	Persist sessions/scores and serve them to the dashboard.
Steps	1. POST /session, /score, /rom 2. GET history for a patient 3. Run locally (uvicorn), CORS for PWA
Dependencies	DB-1
Deliverable	app.py with REST endpoints
Effort	1.5 d

BE-2 Progression rule service

Objective	Compute next difficulty from recent performance.
Steps	1. Read last 2 sessions 2. Apply $\geq 80\%$ correct + pain ≤ 5 rule 3. Return advance / hold / regress
Dependencies	BE-1, DB-1
Deliverable	progression.py endpoint
Effort	1 d

3.4 Database

DB-1 SQLite schema & data layer

HIGH

Objective	Create the relational store for patients, sessions, scores, ROM, progress.
Steps	1. Create 5 tables (see §Database) 2. db.py CRUD helpers 3. Seed + migration script
Dependencies	None
Deliverable	schema.sql + db.py
Effort	1 d

DB-2 Client-side fallback store

Objective	Offline-first storage when backend is absent.
Steps	1. IndexedDB/localStorage wrapper 2. Same shape as API payloads 3. Sync-on-reconnect stub
Dependencies	DB-1
Deliverable	storage.js
Effort	0.5 d

3.5 Dashboard

DASH-1 Progress dashboard

Objective	Weekly ROM, reps, hold-time and adherence charts for patient/clinician.
Steps	1. Fetch history via BE-1 2. Line + bar charts (reuse chart code) 3. Adherence % + streak tiles
Dependencies	BE-1, DB-1
Deliverable	dashboard.html
Effort	1.5 d

3.6 Testing

QA-1 Unit tests — angle & hold logic

Objective	Lock down the math with deterministic tests.
Steps	1. Fixtures of known landmark sets 2. Assert angle, hold, rep outputs 3. CI-runable script
Dependencies	AI-1
Deliverable	test_core.py
Effort	0.5 d

QA-2 Goniometer validation study

Objective	Quantify sensor accuracy across the working range.
Steps	1. Capture vs goniometer, several volunteers 2. Compute MAE + Bland–Altman 3. Write up as methods evidence
Dependencies	MVP build
Deliverable	validation_report.pdf
Effort	1.5 d

3.7 Documentation

DOC-1 Fix README path + user guide

Objective	Correct the run path and add a patient/clinician quick-start.
Steps	1. Fix /mobile_app/ → root 2. One-page setup + camera guide 3. Screenshots
Dependencies	None
Deliverable	Updated README + USERGUIDE.md
Effort	0.5 d

DOC-2 Methods & API documentation

Objective	Document the pipeline and endpoints for the thesis.
Steps	1. Angle method + filtering write-up 2. API reference 3. Data dictionary
Dependencies	BE-1, DB-1
Deliverable	docs/ methods + api
Effort	1 d

3.8 Deployment

DEP-1 PWA release to GitHub Pages

Objective	Ship the game build to the phone-accessible URL.
Steps	1. Enable Pages (main/root) 2. Version + cache-bust assets 3. Smoke test on phone over HTTPS
Dependencies	FE-2
Deliverable	Live PWA URL
Effort	0.5 d

DEP-2 Backend + DB packaging

Objective	Make the local service reproducible.
Steps	1. requirements.txt + run script 2. README run steps 3. Optional single-command launcher
Dependencies	BE-1
Deliverable	Reproducible local deploy
Effort	0.5 d

4

Daily Execution Plan

A baby-step schedule sequencing the roadmap tasks. Days are working days; adjust to your calendar.

Day	Focus	Tasks	Deliverable
Day 1 (Today)	Isometric game vertical slice	AI-1, FE-1, FE-2	Playable hold game (rough)
Day 2	Hold scoring, results, feedback cues	AI-2, AI-3, FE-3	Scored session + cues
Day 3	Database + client store	DB-1, DB-2	Sessions persist
Day 4	Backend API + wire game→DB	BE-1	Saved history via API
Day 5	Dashboard v1	DASH-1	Weekly progress charts
Day 6	Progression + safety (NPRS, caps)	BE-2, safety	Auto-difficulty + pain gate
Day 7	Flagship Bounce-to-Target game	FE-2 reuse	2nd game working
Day 8	Unit tests + bug bash	QA-1	Green tests
Day 9	Docs + README fix + user guide	DOC-1, DOC-2	Documentation set
Day 10	Deploy PWA + backend, phone smoke test	DEP-1, DEP-2	Live MVP
Day 11–12	Goniometer validation study	QA-2	Validation report
Day 13–14	Usability pilot (3–5 users) + fixes	MVP	Pilot feedback + patches

5

Testing Plan

Level	What is tested	Method	Pass criteria
Unit	Angle math, hold, rep, tempo functions	Known-landmark fixtures	Outputs match expected ±tol
Integration	Pose → angle → game → DB path	Live camera / recorded clips	Session logs correctly
Accuracy	Knee angle vs goniometer	Multi-volunteer captures	MAE reported; Bland–Altman
Rep/hold count	Counter vs manual count	Scripted movements	≥95–100% agreement
Usability	End-to-end on target users	3–5 pilot participants	SUS score; task success
Safety	Pain gate, caps, form flags fire	Boundary inputs	No unsafe rep is scored
Cross-device	Phone (PWA) + laptop	iOS/Android/desktop	Runs over HTTPS; acceptable FPS

6

Deployment Plan

6.1 Environments & Targets

Component	Target	Notes	Release step
Frontend (PWA)	GitHub Pages (HTTPS)	Static build; installable; phone camera	git push → Pages
Backend API	Local host (uvicorn)	FastAPI + SQLite for sessions	run.sh / uvicorn app:app
Database	Local SQLite file	Zero-setup; file-backed	Auto-created on first run
Dashboard	Served by backend / static	Reads history API	Bundled with backend

6.2 Release Checklist

- Version bumped; assets cache-busted
- PWA loads over HTTPS on a real phone; camera permission works
- Model + WASM load from CDN with CPU fallback verified

- Backend starts with one command; SQLite auto-creates
- Sessions persist and appear in the dashboard
- Smoke test: full game → results → export → history
- README run steps validated on a clean machine

7 Final Deliverables

Deliverable	Description	Source task
Isometric rehab game	Playable, scored, safe isometric hold game	§ High-Priority
Flagship game	Bounce-to-Target sit-to-stand game	FE-2 reuse
Feedback engine	Real-time corrective cues + tempo gate	AI-2/3
Database	SQLite store + data layer	DB-1
Backend API	FastAPI session/score/progression service	BE-1/2
Dashboard	Weekly progress + adherence view	DASH-1
Validation report	MAE + Bland–Altman vs goniometer	QA-2
Documentation	README, user guide, methods, API docs	DOC-1/2
Deployed MVP	Live PWA + reproducible backend	DEP-1/2
Pilot results	Usability + bug-fix log (3–5 users)	Day 13–14

8 Timeline & Milestones

Milestone	Duration	Dependencies	Deliverable
M1 · Isometric game MVP	Days 1–2	AI-1, FE-1/2	Playable scored hold game
M2 · Data layer live	Days 3–4	DB-1, BE-1	Sessions persisted via API
M3 · Dashboard + progression	Days 5–6	DASH-1, BE-2	Progress view + auto-difficulty + safety
M4 · Second game + tests	Days 7–8	FE-2, QA-1	2 games, green unit tests
M5 · Docs + deploy	Days 9–10	DOC, DEP	Live MVP + documentation
M6 · Validation	Days 11–12	QA-2	Goniometer validation report
M7 · Pilot	Days 13–14	MVP	Usability pilot complete
M8 · RCT + thesis	Weeks 5–8+	Ethics approval	Data collection, analysis, write-up

★ High Priority (Today) — Isometric Gamified Rehab Game

Working title: Wall-Sit Guardian. The patient holds a safe knee-flexion angle (isometric quadriceps hold); staying inside the target band charges an on-screen shield/beam. Drift out of band breaks the hold. This proves the full movement-to-game loop on the existing pose engine.

1 • Clinical Goal

Aspect	Detail
Rehabilitation objective	Build quadriceps isometric endurance and neuromuscular control at a safe, pain-free knee angle — the earliest-phase, lowest-load KOA strengthening option.
Why isometric	Isometrics reduce pain and build strength with minimal joint movement — ideal for irritable knees and early rehab.
Safety considerations	Angle band capped within prescribed safe ROM; back-supported wall-sit; pre/post NPRS pain check (≥5/10 → stop & rest); max hold + daily caps; support within reach; drift/instability breaks hold (no unsafe scoring).

2 • Game Design

Element	Design
Core gameplay	Hold the knee inside a target angle band to keep a shield/beam charged; the longer and steadier the hold, the stronger it grows.
Exercise flow	Calibrate → position (wall-sit) → 3-2-1 → hold for target seconds → rest → repeat for prescribed sets.
Scoring	Points ∝ time-in-band; steadiness bonus for low angle variance; a hold only counts if it meets minimum duration and stays in band.
Feedback	Live: 'bend a little more / ease up / hold steady'. End-of-hold: correct % + why. Colour band: green in-zone, amber near edge.
Rewards	Stars per set (quality-based), streak badges for adherence days, shield 'tier' upgrades as endurance improves.
Progress tracking	Best hold, total hold time, in-band %, per-session trend; feeds ROM/endurance history and dashboard.

3 • Technical Design

Layer	Approach
UI screens	Menu → Calibrate → Game (HUD + hold gauge) → Results.
Pose integration	Reuse MediaPipe Pose + existing view-adaptive knee-angle engine and calibration ('Cal 0°').
Isometric hold detection	Target band [θ -tol, θ +tol]; accumulate time-in-band; reset on drift > tol; steadiness = rolling angle variance.
Data storage	Write hold events + summary to SQLite via backend (DB-1/BE-1); client-side fallback store when offline.
Progress reporting	Results screen + CSV/JSON export; history + charts surfaced in the dashboard (DASH-1).

4 • Development Plan (implementation steps)

Each step: objective · files/modules · output.

Step	Objective	Files / modules	Output
1 • Wireframes	Sketch the 4 screens & HUD	(design) wireframes.png	Agreed layout
2 • UI	Build screens & navigation	index.html, game_isometric.css	Static screens navigable
3 • Game loop	raf loop, states (idle/hold/rest)	game_isometric.js	Running loop + timer
4 • Pose detection	Feed angle from existing engine	(reuse) pose+angle in index.html	Live knee angle in game
5 • Hold detection	Time-in-band + drift reset	holdDetector.js	Hold progress 0–1
6 • Scoring	Points, steadiness bonus, gating	scoring.js	Scored, gated holds
7 • Results screen	Summary + export + save	results.js	Session summary + export
8 • Testing	Unit + on-device check	test_core.py	Verified behaviour
9 • Integration	Wire to DB/API + dashboard	db.js, app.py, dashboard.html	End-to-end persisted game

Timeline — Isometric Game

Task	Duration	Dependencies	Deliverable
Wireframes + UI	2–3 h	None	Navigable screens
Game loop + pose	2–3 h	UI	Live angle in loop
Hold + scoring	2–3 h	Loop	Scored holds
Results + integration	2–3 h	DB-1	Persisted session
Testing	1–2 h	All	Verified game

✓ Final Checklist — Remaining Deliverables

Everything outstanding before project completion.

Build

- Isometric hold game — playable, scored, safe (TODAY)
- Flagship Bounce-to-Target sit-to-stand game
- Coaching feedback engine + tempo gate + trunk-lean detection
- Safety layer: NPRS pain check, hold/depth caps, daily caps
- SQLite database (patients, sessions, scores, ROM, progress)
- Backend API (FastAPI) persisting and serving sessions
- Progression rule (auto difficulty) service
- Progress dashboard (ROM, reps, hold-time, adherence)

Quality & Docs

- Unit tests passing for angle / hold / rep / tempo logic
- Goniometer validation report (MAE + Bland–Altman)
- Rep/hold-count accuracy ≥95–100% vs manual
- README path fixed + patient/clinician user guide
- Methods chapter + API documentation

Release & Research

- PWA deployed to GitHub Pages; phone smoke test passed
- Backend + DB reproducible from README on a clean machine
- Usability pilot (3–5 users) completed and issues triaged
- Ethics approval obtained (critical path)
- RCT run (both arms) — WOMAC, NPRS, TUG, chair-stand, ROM, Berg
- Data analysis, discussion, limitations & future work written

End of execution plan · MoveLab / KneeQuest · Prepared for supervisor review · 25 July 2026